

FELHANTERING I PYTHON

ValueError och ZeroDivisionError

Programmering 1 – JENSEN

Lärare: George Glor

1. Vad händer när det blir fel?

När Python stöter på ett fel kraschar programmet och slutar köra. Användaren ser ett felmeddelande.

Exempel – användaren skriver text istället för tal:

```
tal = int(input("Ange ett tal: "))  
# Användaren skriver: hej
```

Felmeddelande:

```
ValueError: invalid literal for int()  
with base 10: 'hej'
```

Programmet kraschar! Allt efter den raden körs inte.

Exempel – division med noll:

```
resultat = 100 / 0
```

Felmeddelande:

```
ZeroDivisionError: division by zero
```

Vi kan fånga dessa fel med try/except så att programmet fortsätter istället för att krascha.

2. ValueError – fel typ av värde

ValueError uppstår när Python försöker konvertera ett värde till fel typ. Vanligaste orsaken: int() eller float() får text som inte är ett tal.

Dessa ger ValueError:

```
int("hej")      # ValueError!  
int("abc")      # ValueError!  
int("")         # ValueError!  
float("text")   # ValueError!
```

Dessa fungerar:

```
int("42")       = 42  
int("-5")       = -5  
float("3.14")   = 3.14
```

Lösning: Använd try/except för att fånga felet:

```
try:  
    tal = int(input("Ange ett tal: "))  
    print("Du angav:", tal)  
except ValueError:  
    print("Det där var inte ett tal!")
```

Resultat:

```
Ange ett tal: hej  
Det där var inte ett tal!
```

3. ZeroDivisionError – division med noll

ZeroDivisionError uppstår när du försöker dividera med 0. Det är matematiskt omöjligt.

Dessa ger ZeroDivisionError:

```
10 / 0      # ZeroDivisionError!
100 / 0     # ZeroDivisionError!
5 // 0      # ZeroDivisionError!
7 % 0       # ZeroDivisionError!
```

Dessa fungerar:

```
10 / 2      = 5.0
10 / 0.5     = 20.0
0 / 10       = 0.0   (0 delat med något = 0)
```

Lösning:

```
try:
    tal = int(input("Ange ett tal: "))
    resultat = 100 / tal
    print("100 /", tal, "=", resultat)
except ZeroDivisionError:
    print("Kan inte dividera med noll!")
```

Resultat:

```
Ange ett tal: 0
Kan inte dividera med noll!
```

4. Hur try/except fungerar

Struktur:

```
try:
    # Koden som KAN ge fel
    ...
except FelTyp:
    # Koden som körs OM felet uppstår
    ...
```

- **try:** Python försöker köra koden
- Om det går bra → except hoppas över
- Om fel uppstår → Python hoppar till except
- Programmet kraschar inte – det fortsätter

5. Fånga båda felen samtidigt

Båda felen:

```
try:
    tal = int(input("Ange ett tal: "))
    resultat = 100 / tal
    print("100 /", tal, "=", resultat)
except ValueError:
    print("Skriv ett tal, inte text!")
except ZeroDivisionError:
    print("Kan inte dela med noll!")
```

Varje fel får sitt eget meddelande. Användaren vet exakt vad som gick fel.

Testa själv:

- Skriv "hej" → ValueError → "Skriv ett tal, inte text!"
- Skriv "0" → ZeroDivisionError → "Kan inte dela med noll!"
- Skriv "5" → Inget fel → "100 / 5 = 20.0"

6. Fråga tills rätt svar

Kombinera try/except med en while-loop för att fråga tills användaren ger rätt input:

```
while True:
    try:
        tal = int(input("Ange ett heltal: "))
        break # Rätt input → avsluta loopen
    except ValueError:
        print("Fel! Skriv ett heltal.")
print("Du angav:", tal)
```

Resultat:

```
Ange ett heltal: hej
Fel! Skriv ett heltal.
Ange ett heltal: abc
Fel! Skriv ett heltal.
Ange ett heltal: 42
Du angav: 42
```

Mönstret: while True + try + break = fråga tills det fungerar.

7. Komplette exempel

Säker division med loop:

```
while True:
    try:
        tal1 = float(input("Tal 1: "))
        tal2 = float(input("Tal 2: "))
        resultat = tal1 / tal2
        print(f"{tal1} / {tal2} = {resultat}")
        break
    except ValueError:
        print("Skriv ett giltigt tal!")
    except ZeroDivisionError:
        print("Tal 2 kan inte vara noll!")
```

Resultat:

```
Tal 1: 10
Tal 2: hej
Skriv ett giltigt tal!
Tal 1: 10
Tal 2: 0
Tal 2 kan inte vara noll!
Tal 1: 10
Tal 2: 4
10.0 / 4.0 = 2.5
```

PROV – Felhantering

Fråga 1: Vad ger `int("hej")`?

- a) 0
- b) "hej"
- c) ValueError

Svar: c) ValueError

"hej" är inte ett tal. `int()` kan bara konvertera strängar med siffror.

Fråga 2: Vad ger `10 / 0`?

- a) 0
- b) 10
- c) ZeroDivisionError

Svar: c) ZeroDivisionError

Division med noll är matematiskt omöjligt.

Fråga 3: Vad gör `try/except`?

- a) Fixar felet automatiskt
- b) Fångar felet så programmet inte kraschar
- c) Tar bort felaktig kod

Svar: b) Fångar felet så programmet inte kraschar

`try` försöker köra koden. Om fel → `except` körs. Programmet fortsätter.

Fråga 4: När uppstår ValueError?

- a) När du delar med 0
- b) När du konverterar text till tal
- c) När du glömmer kolon

Svar: b) När du konverterar text till tal

`int("hej")` → ValueError. `int("42") = 42` fungerar.

Fråga 5: Vad skriver koden ut? `try: print(10/5) except ZeroDivisionError: print("Fel!")`

- a) 2.0
- b) Fel!
- c) Inget

Svar: a) 2.0

`10/5 = 2.0` (inget fel). `except` hoppas över.

Fråga 6: Vad skriver koden ut? `try: print(10/0) except ZeroDivisionError: print("Fel!")`

- a) 0
- b) Fel!
- c) Programmet kraschar

Svar: b) Fel!

`10/0` → ZeroDivisionError. `except` fångar felet och skriver "Fel!"

Fråga 7: Vad ger 0 / 10?

- a) 0.0
- b) ZeroDivisionError
- c) ValueError

Svar: a) 0.0

0 delat med något = 0. Det är bara något delat med 0 som ger fel.

Fråga 8: Vad ger int("3.14")?

- a) 3
- b) 3.14
- c) ValueError

Svar: c) ValueError

int() kan inte konvertera decimaltal som sträng. Använd float("3.14") eller int(float("3.14")).

Programmering 1 på JENSEN. Lärare: George Glor.